Communication Efficient Gradient Coding: A Survey

Jay Mehta: 22B1281
Kshitij Vaidya: 22B1829
Tanay Bhat: 22B3303

May 5, 2026

Contents

Chapter 1

Introduction

Distributed learning helps address the challenges faced my large-scale machine learning tasks by outsourcing its computations to worker nodes. Such a system is susecptible to communication bottenecks caused by end-to-end delays between the workers and the central server. Another possible issue is the presence of stragglers (nodes that are slow to respond). In this scenario, the central server does not have enough data to compute the gradient to update its model parameters. One way to mitigate this is to split the dataset into k smaller batches and assign d batches to each of the n workers. The i-th worker then sends a function $f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d})$ of the gradients to the central server, where g_{i_j} is the gradient computed by the worker on the j-th batch assigned to it. The server is able to recover the full gradient even if some of the workers are stragglers, as long as it receives enough h_i's. We shall now formally define the problem of gradient coding and then discuss a communication efficient scheme to solve it.

1.1 Problem Statement

Consider a dataset $D = \{(x_i, y_i)\}_{i=1}^N$ where $x_i \in \mathbb{R}^l, y_i \in \mathbb{R}$. Our goal is to minimize the loss function $L(D; w) = \sum_{i=1}^N L(w; x_i, y_i)$ where $w \in \mathbb{R}^d$ is the model parameter. One way to do this to perform:

$$w^{(t+1)} = h(w^{(t)}, g^{(t)}) \tag{1.1}$$

Where $g^{(t)} = \sum_{i=1}^N \nabla L(w^{(t)}; x_i, y_i)$ and h is some gradient based update. In a distributed setting, we assume n workers $W_1, W_2, \ldots, W_n$ and split the dataaset D into k equal batches $D_1, D_2, \ldots, D_k$. Each worker is assigned d batches and computes the following:

$$g_i^{(t)} = \sum_{(x,y) \in D_i} \nabla L(w^{(t)}; x, y) \tag{1.2}$$

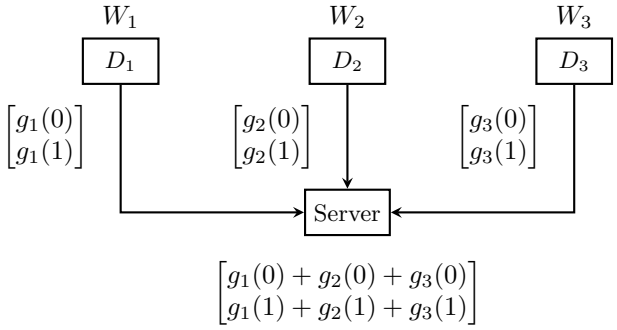
Hence we see that $g^{(t)} = \sum_{i=1}^{k} g_i^{(t)}$. We assume that s worker nodes are stragglers and hence we only have access to $n - s$ of the $g_i^{(t)}$'s. Say for each$i \in 1, 2, \ldots n$ we represent the datasets assigned to W_i as $\{D_{i_1}, D_{i_2}, \ldots, D_{i_d}\}$. The i-th worker then sends a function $f_i(g_{i_1}^{(t)}, g_{i_2}^{(t)}, \ldots, g_{i_d}^{(t)})$ of the partial gradients. This paper only considers linear functions of the form:

$$f_i(g_{i_1}^{(t)}, g_{i_2}^{(t)}, \ldots, g_{i_d}^{(t)}) = \sum_{j=1}^{d} a_{i_j} g_{i_j}^{(t)} \tag{1.3}$$

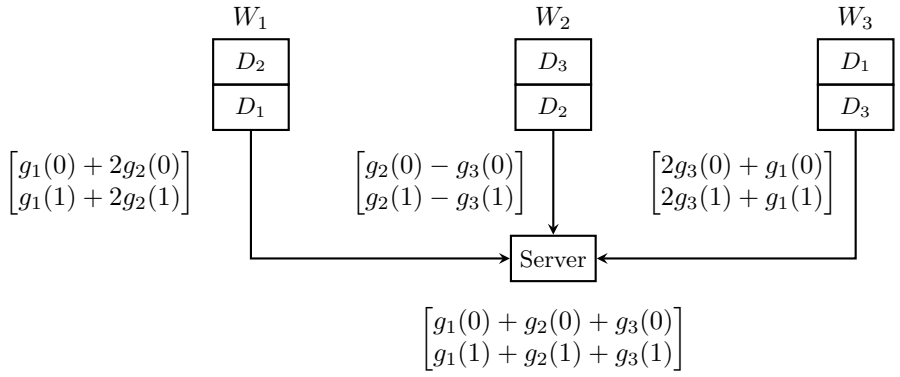
The partial gradient vector $g_i^{(t)} = [g_i^{(t)}(0), g_i^{(t)}(1), \ldots, g_i^{(t)}(l-1)]$ may require a large value of l to be tolerant to stragglers or use a smaller l but have less tolerance to stragglers. We shall drop the superscript (t) for simplicity.

1.2 Tolerance-Efficiency Tradeoff

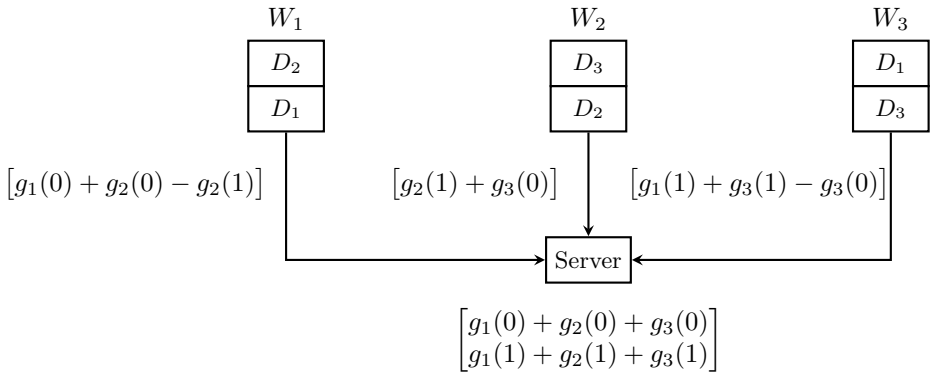
Consider the scenario where $n = 3, k = 3$ we can have the following assignment of batches to workers:



This is a basic scheme and the server needs to wait for all of the workers to respond before it can compute the full gradient. Such a scheme is neither communication efficient nor straggler tolerant. Now consider the following assignment of batches to workers:



Here the server can compute the full gradient even if one of the workers is a straggler. If W_1 is a straggler, the server can compute the full gradient as $g = f_1 - f_2$, if W_2 is a straggler, the server can compute the full gradient as $g = f_2 + f_3$ and if W_3 is a straggler, the server can compute the full gradient as $g = \frac{1}{2}(f_3 + f_1)$. However, this scheme is not communication efficient as each worker sends a vector of size l. Consider:



In this case $l = 1$ and the server computes the full gradient as $g(0) = f_1 + f_2$ and $g(1) = f_2 + f_3$. However, this scheme is not straggler tolerant as the server needs to wait for all the workers to respond before it can compute the full gradient. Hence we see that there is a tradeoff between straggler tolerance and communication efficiency. We shall now proceed with the main result presented by the paper and then discuss the scheme that achieves it.

Chapter 2

Main Theorem

2.1 Main Result

Theorem 1. *Given integers n, k a triple (d, s, m) where $1 \leq d \leq k$ and $m \geq 1$ is achievable if there is a distributed synchronous gradient coding scheme with n workers and k data batches such that:*

1. *Each worker is assigned d data batches*
2. *There are n functions $f_i : \mathbb{R}^{dl} \to \mathbb{R}^{l/m}$ such that the server can recover the full gradient g from any $n - s$ of the vectors $f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d})$ where $i_1, i_2, \ldots, i_d$ are the indices of the data batches assigned to worker W_i.*
3. *Each f_i is a linear combination of the partial gradients $g_{i_1}, \ldots, g_{i_d}$*

and such a triple is achievable if and only if the following condition holds:

$$\frac{d}{k} \geq \frac{s+m}{n} \tag{2.1}$$

We shall first show that any achievable (d, s, m) must satisfy the above condition and then present the paper's coding scheme for $n = k$ that satisfies $d \geq s + m$ with equality.

2.2 Forward Direction

We assume that the triple (d, s, m) is achievable and show that it must satisfy the condition $\frac{d}{k} \geq \frac{s+m}{n}$. Before wew proceed, we prove the following claim:

Claim 1. *For each $i \in [k]$, data batch D_i is assigned to at least $s + m$ workers.*

Proof. Say for some $j \in [k]$ data batch D_j is assigned to $a < s + m$ workers. WLOG assume these workers are $W_1, W_2, \ldots, W_a$ and suppose that first s of these workers are stragglers. Based on our definition of achievability, the server should be able to recover the full gradient $g = g_1 + \ldots + g_k$ from the vectors $f_{s+1}, f_{s+2}, \ldots, f_n$. ■

2.3 Coding Scheme